Documentation Externe de l'API Gateway pour les Clients Partenaires

Othmane DERRAR

23 juillet 2024

Table des matières

1 Résumé du Projet **2**

2 Points de Terminaison **2**
2.1 Obtenir la Clé Publique 2
2.2 Authentifier le Client 2
2.3 Renouveler la session 3

3 Sécurité et Implémentation **4**
3.1 Génération et Gestion des Clés RSA 4
3.2 Déchiffrement et Vérification des Hashes 4
3.3 Génération et Gestion des JWT 4
3.4 Validation des JWT 5
3.5 Limitation du Taux (Rate Limiting) 5

4 Exemples d'Utilisation **5**
4.1 Obtenir la Clé Publique 5
4.2 Authentifier le Client 6

5 Gestion des Erreurs **6**
5.1 Réponses d'Erreur 6

1 Résumé du Projet

Le projet consiste à développer un micro-service et une API Gateway d'authentification sécurisée pour les clients partenaires, utilisant le chiffrement RSA pour la sécurité et la tokenisation JWT pour la gestion des sessions.

2 Points de Terminaison

2.1 Obtenir la Clé Publique

Endpoint : `/api/partners-auth/getKey`
Méthode : `POST`
Description : Fournit la clé publique RSA au client pour le chiffrement de son hash.
Exemple de Requête :

```
POST /api/partners-auth/getKey HTTP/1.1
Host: {api-gateway-ip}
Content-Type: application/json
```

Réponse :
— **200 OK** :

```
{
  "publicKey": "string"
}
```

2.2 Authentifier le Client

Endpoint : `/api/partners-auth/authenticate`
Méthode : `POST`
Description : Authentifie le client en vérifiant le hash chiffré du client et, après vérification réussie, fournit un JWT.
Exemple de Requête :

```
POST /api/partners-auth/authenticate HTTP/1.1
Host: {api-gateway-ip}
Content-Type: application/json
```

Body :

```
{
  "partnerId": "string",
  "encryptedHash": "string"
}
```

Réponse :

— **200 OK** :

```
{
  "error": false,
  "message": "Partner is successfully authenticated",
  "token": "string JWT",
  "refreshToken": "string"
}
```

— **404 Not Found** : Hash incorrect ou authentification échouée.

```
{
  "objectName": "partner",
  "message": "Invalid partner Hash"
}
```

2.3 Renouveler la session

Endpoint : `/api/partners-auth/refresh`
Méthode : `POST`
Description : Renouveler la session en vérifiant le **refreshToken** du client et, après vérification réussie, fournit un nouveau **JWT** et **refreshToken**.

Exemple de Requête :

```
POST /api/partners-auth/refresh HTTP/1.1
Host: {api-gateway-ip}
Content-Type: application/json
```

Header :

```
Authorization: "Bearer string"
```

Body :

```
{
  "refreshToken": "string"
```

```
}
```

Réponse :

— **200 OK** :

```
{
  "error": false,
  "message": "Partner is successfully authenticated",
  "token": "string JWT",
  "refreshToken": "string"
}
```

— **404 Not Found** : Hash incorrect ou authentification échouée.

```
{
  "objectName": "partner",
  "message": "Invalid partner Hash"
}
```

3 Sécurité et Implémentation

3.1 Génération et Gestion des Clés RSA

— Utilisation de la Java Cryptography Architecture (JCA) pour générer des paires de clés publiques/privées RSA.
— La clé privée est stockée en toute sécurité et n'est accessible qu'à l'API d'authentification.

3.2 Déchiffrement et Vérification des Hashes

— Le hash chiffré reçu du client est déchiffré en utilisant la clé privée stockée.
— Le hash déchiffré est comparé au hash partenaire stocké pour vérification.

3.3 Génération et Gestion des JWT

— Après la vérification réussie du hash du client, un JWT est généré.
— Le JWT contient des informations telles que l'ID du partenaire et le hash du partenaire, et il est signé en utilisant un algorithme HMAC sécurisé (HS256).

3.4 Validation des JWT

— Une méthode est mise en place pour valider les JWT entrants.
— Le token n'est pas expiré et la signature est valide. Les tokens invalides sont rejetés.

3.5 Limitation du Taux (Rate Limiting)

— Les points de terminaison de l'API sont soumis à des limites de taux pour éviter les abus et garantir une utilisation équitable.
— Les limites sont configurées comme suit :
  — /**api**/**partners-auth**/**getKey** : 8 requêtes par minute (soit 1 requête par 7.5 secondes)
  — /**api**/**partners-auth**/**authenticate** : 12 requêtes par minute (soit 1 requête par 5 secondes)
  — /**api**/**partners-auth**/**refresh** : 40 requêtes par minute (soit 1 requête par 1.5 secondes)
  — /**api**/**partners-auth**/**validateToken** : 80 requêtes par minute (soit 1 requête par 0.75 secondes)
— En cas de dépassement de la limite de taux, une réponse HTTP 403 est renvoyée avec le message suivant :

```
{
  "error": "Too many requests",
  "message": "Try again later"
}
```

4 Exemples d'Utilisation

4.1 Obtenir la Clé Publique

Requête :

```
POST /api/partners-auth/getKey HTTP/1.1
Host: api-gateway-ip
Content-Type: application/json
```

Body :

```
{
  "partnerHash": "string"
```

```
}
```

Réponse :

```
{
  "publicKey": "string"
}
```

4.2 Authentifier le Client

Requête :

```
POST /api/partners-auth/authenticate HTTP/1.1
Host: api-gateway-ip
Content-Type: application/json
```

Body :

```
{
  "partnerId": "partner123",
  "encryptedHash": "base64EncryptedHash"
}
```

Réponse :

```
{
  "token": "string JWT"
}
```

5 Gestion des Erreurs

5.1 Réponses d'Erreur

— **401 Unauthorized** : Authentification échouée, hash incorrect ou token invalide.

```
{
  "error": "true",
  "message": "Invalid JWT token"
}
```

— **403 Forbidden** : Limite de taux dépassée.

```
{
  "error": "Too many requests",
  "message": "Try again later"
}
```

Cette documentation fournit les informations nécessaires pour que les équipes de développement des clients partenaires puissent intégrer leurs applications avec l'API Gateway et le service d'authentification sécurisé. Pour toute question ou assistance, veuillez contacter notre support technique.